Domain and Range

My Definition

Facts & Characteristics

Domain

Examples

Non-examples

My Definition

Facts & Characteristics

Range

Examples

Non-examples

Contracts

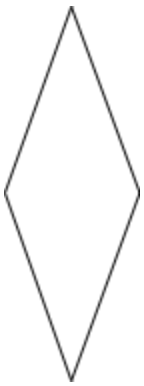
Contracts tell us how to use a function. For example: `ellipse :: (Number, Number, String, String) -> Image` tells us that the name of the function is `ellipse`, it takes four inputs (two Numbers and two Strings), and it evaluates to an `Image`. From the contract, we know `ellipse(50, 100, "solid", "teal")` will evaluate to an `Image`.

Name		Domain		Range
`# triangle`	`::`	`Number, String, String`	`->`	`Image`
`triangle(80, "solid", "darkgreen")`				
`# star`	`::`	`Number, String, String`	`->`	`Image`
`star(50, "solid", "teal")`				
`# circle`	`::`	`Number, String, String`	`->`	`Image`
`circle(30, "outline", "fuchsia")`				
`# square`	`::`	`Number, String, String`	`->`	`Image`
`square(10, "outline", "red")`				
`# rectangle`	`::`	`Number, Number, String, String`	`->`	`Image`
`rectangle(20, 80, "solid", "gold")`				
`# rhombus`	`::`	`Number, Number, String, String`	`->`	`Image`
`rhombus(20, 50, "solid", "turquoise")`				
`# ellipse`	`::`	`Number, Number, String, String`	`->`	`Image`
`ellipse(30, 70, "outline", "lightblue")`				
`# text`	`::`	`String, Number, String`	`->`	`Image`
`text("I'm thankful for…", 50, "green")`				
`# regular-polygon`	`::`	`Number, Number, String, String`	`->`	`Image`
`regular-polygon(8, 40, "solid", "red")`				
`#right-triangle`	`::`	`Number, Number, String, String`	`->`	`Image`

Using Contracts (continued)

Use the contracts to write expressions to generate images similar to those pictured.

`rhombus :: Number, Number, String, String -> Image`

	`rhombus(100, 140, "solid", "black")`
	`rhombus(100, 40, "outline", "black")`
Write an expression for a square (rotated) using `rhombus`!	`rhombus(100, 90, "solid", "black")`
What variable changes with the first Number?	*The length of each side*
What variable changes with the second Number?	*The top and bottom angle of the rhombus*

Triangle Contracts (SAS & ASA)

Type each expression (left) below into the editor and match it to the image it creates (right).

Expression			Image
`triangle-sas(120, 45, 70, "solid", "black")`	1	A	
`triangle-sas(120, 90, 70, "solid", "black")`	2	B	
`triangle-sas(120, 135, 70, "solid", "black")`	3	C	
`triangle-sas(70, 135, 120, "solid", "black")`	4	D	

Think about how you would describe each of the arguments that `triangle-sas` takes in to someone who'd never used the function before and annotate the contract below using descriptive variable names.

```
triangle-sas :: (
    ____________________side1____________________ :: Number,
    ____________________angle____________________ :: Number,
    ____________________side2____________________ :: Number,
    ____________________style____________________ :: String,
    ____________________color____________________ :: String
) -> Image
```

Add examples of each of the triangle functions we've explored to your contracts page.

If you have time, experiment with the `triangle-asa` function.

```
triangle-asa :: (

    left-angle :: Number,
    left-side :: Number,
    bottom-angle :: Number,
    style :: String
    color :: String
) -> Image
```

Star Polygon

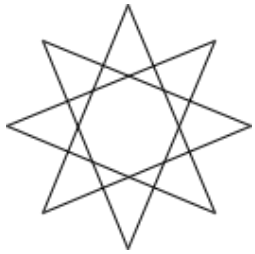
```
star-polygon :: (

    side-length :: Number,
    points-on-polygon :: Number,
    polygon-points-to-skip-between-star-points :: Number,
    shading-style  :: String,
    color :: String

) -> Image
```

Using the detailed contract above, write expressions to create each image below.

Then make two more star polygons of your choosing. Sketch them and write expressions to generate them.

	`star-polygon(50, 8, 3, "outline", "black")`
	`star-polygon(50, 8, 2, "solid", "black")`

Mapping Examples with Circles of Evaluation

Contract: _____

Purpose Statement: _____

If I type…	→	It should map to…
EXAMPLE #1: Circle of Evaluation Code:	→	Circle of Evaluation: Code:
EXAMPLE #2: Circle of Evaluation Code:	→	Circle of Evaluation: Code:

Do The Examples Have the Same Contracts?

For each pair of Examples below, decide whether the two examples have the same contract. If they do, fill in the contract in the space provided. If not, write a few words explaining how you know their contracts aren't the same.

1) no match (the range types are different)

```
examples:
  mystery(30) is 30 * 50
  mystery(10) is text("Welcome!", 10, "darkgreen")
end
```

2) # mystery :: Number, Number -> Number

```
examples:
  mystery(30, 40) is 40 - (2 * 30)
  mystery(10, 15) is 15 - (2 * 10)
end
```

3) no match (the domain types are different)

```
examples:
  mystery("New York") is text("New York", 20, "red")
  mystery(20) is text("New York", 20, "red")
end
```

4) no match (the domain types are in different orders)

```
examples:
  mystery("green", 32) is circle(32, "outline", "green")
  mystery(18, "green") is circle(18, "outline", "green")
end
```

5) no match (the domains have different numbers of arguments)

```
examples:
  mystery(6, 9, 10) is 6 / (9 + 10)
  mystery(3, 7) is 3 / (7 + 10)
end
```

6) # mystery :: String, String -> Image

```
examples:
  mystery("red", "blue") is text("blue", 25, "red")
  mystery("purple", "Go Team!") is text("Go Team!", 25, "purple")
end
```

Do The Examples Have the Same Contracts? (2)

For each pair of Examples below, decide whether the two examples have the same contract. If they do, fill in the contract in the space provided. If not, write a few words explaining how you know their contracts aren't the same.

1) `# mystery :: Image -> Image`

```
examples:
  mystery(triangle(70, "solid", "green")) is
    triangle(140, "solid", "green")
  mystery(circle(100, "solid", "blue")) is
    circle(200, "solid", "blue")
end
```

2) no match (the domains have different numbers of arguments)

```
examples:
  mystery("red") is triangle(140, "solid", "red")
  mystery("blue", "circle") is circle(140, "solid", "blue")
end
```

3) no match (the domains have different numbers of arguments)

```
examples:
  mystery("+", 4, 5) is 4 + 5
  mystery("sqrt", 25) is num-sqrt(25)
end
```

4) `# mystery :: String, Number -> Number`

```
examples:
  mystery("circle", 4) is num-pi * num-sqr(4)
  mystery("square", 5) is num-sqr(5)
end
```

5) no match (the range types are different)

```
examples:
  mystery("dog") is 3
  mystery("cat") is "kitten"
end
```

6) `# mystery :: String -> Number`

```
examples:
  mystery("dog") is 3
  mystery("kitten") is 6
end
```

Matching Contracts and Examples

Match each Example on the left with its Contract on the right. NOTE: Multiple examples may match to the same contract!

Contract			Examples

`examples:`
`  match(circle(10, "solid", "green")) is`
`  rotate(37, circle(10, "solid", "green"))`
`end`
	1-C	A	`# match :: Number, Image -> Image`

`examples:`
`  match(triangle(20, "solid", "blue"), 3) is`
`  scale(3, triangle(20, "solid", "blue"))`
`end`
	2-B		

`examples:`
`  match(circle(20, "outline", "gold")) is`
`  rotate(37, circle(20, "outline", "gold"))`
`end`
	3-C	B	`# match :: Image, Number -> Image`

`examples:`
`  match(30, "red") is 30 + string-length("red")`
`end`
	4-D		

`examples:`
`  match(circle(10, "solid", "orange"), 22) is`
`  scale(22, circle(10, "solid", "orange"))`
`end`
	5-B		

`examples:`
`  match(10, "blue") is 10 + string-length("blue")`
`end`
	6-D	C	`# match :: Image -> Image`

`examples:`
`  match(5, star(20, "solid", "red")) is`
`  rotate(90 - 5, star(20, "solid", "red"))`
`end`
	7-A		

`examples:`
`  match(num-abs(-4), "45") is 4`
`end`
	8-D	D	`# match :: Number, String -> Number`

Matching Contracts and Examples 2

Match each Example on the left with its Contract on the right. NOTE: Multiple examples may match to the same contract!

Contract		Examples	

`examples:`
`  match(1.5) is "greater than 1"`
`end`
	1-A		

`examples:`
`  match(24) is star(24 * 2, "outline", "purple")`
`end`
	2-B		

`examples:`
`  match(string-length("tabletop")) is "8"`
`end`
	3-A	A	`# match :: Number -> String`

`examples:`
`  match(star(20, "outline", "red"), 3) is`
`  3 * image-height(star(20, "outline", "red"))`
`end`
	4-E	B	`# match :: Number -> Image`

`examples:`
`  match(circle(10, "solid", "silver"), 16) is`
`  16 * image-height(circle(10, "solid", "silver")`
`)`
`end`
	5-E	C	`# match :: Number, Number -> Number`

`examples:`
`  match("triangle", "blue") is triangle(40,`
`"outline", "blue")`
`end`
	6-D	D	`# match :: String, String -> Image`

`examples:`
`  match(30) is star(30 * 2, "outline", "purple")`
`end`
	7-B	E	`# match :: Images, Number -> Number`

`examples:`
`  match(string-length("coffee"), string-length(`
`"tea")) is`
`  6 + 3`
`end`
	8-C		